

Networking

An application can *talk* to the outside world through the use of SMS messaging and emails. Another way to communicate with the outside world is through the wireless network available on your Android device. We can use the HTTP protocol to talk to web servers so that text and binary data you can be download. XML files can be parsed to extract the relevant parts of an XML document— a technique that is useful if you are accessing *web services*.

An Android application can also be used to connect to servers using TCP sockets. Using sockets programming, you can write sophisticated, interesting networked applications.

1. Consuming Web Services using HTTP

One common way to communicate with the outside world is through HTTP. HTTP is no stranger to most people; it is the protocol that drives much of the web's success. Using the HTTP protocol, you can perform a variety of tasks, such as downloading web pages from a web server, downloading binary data, and more.

The following code illustrates how to use the HTTP protocol to connect to the web to download all sorts of content.

AndroidManifest.xml

```
...  
<uses-permission android:name="android.permission.INTERNET"/>  
...
```

Java Code

```
...  
  
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
  
    private InputStream OpenHttpConnection(String urlString) throws IOException  
    {  
        InputStream in = null;  
        int response = -1;  
  
        URL url = new URL(urlString);  
        URLConnection conn = url.openConnection();  
  
        if (!(conn instanceof HttpURLConnection))
```

```

        throw new IOException("Not an HTTP connection");
    try{
        HttpURLConnection httpConn = (HttpURLConnection) conn;
        httpConn.setAllowUserInteraction(false);
        httpConn.setInstanceFollowRedirects(true);
        httpConn.setRequestMethod("GET");
        httpConn.connect();
        response = httpConn.getResponseCode();
        if (response == HttpURLConnection.HTTP_OK) {
            in = httpConn.getInputStream();
        }
    }
    catch (Exception ex)
    {
        Log.d("Networking", ex.getLocalizedMessage());
        throw new IOException("Error connecting");
    }
    return in;
}

}
...

```

How does it work?

The application needs the INTERNET permission because the HTTP protocol is used to connect to the web,

The *OpenHttpConnection()* method takes a URL string and returns an *InputStream* object. Using an *InputStream* object, data can be download by reading bytes from the stream object. In this method, you make use of the *HttpURLConnection* object to open an HTTP connection with a remote URL. You set all the various properties of the connection, such as the request method, and so on:

```

        HttpURLConnection httpConn = (HttpURLConnection) conn;
        httpConn.setAllowUserInteraction(false);
        httpConn.setInstanceFollowRedirects(true);
        httpConn.setRequestMethod("GET");

```

After trying to establish a connection with the server, the HTTP response code is returned. If the connection is established (via the response code *HTTP_OK*), then you proceed to get an *InputStream* object from the connection:

```

        httpConn.connect();
        response = httpConn.getResponseCode();
        if (response == HttpURLConnection.HTTP_OK) {
            in = httpConn.getInputStream();
        }

```

Using the *InputStream* object, you can then start to download the data from the server.

2. Downloading Binary Data

A common task you need to perform is downloading binary data from the web. For example, you might want to download an image from a server so that you can display it in your application. The following code illustrates how this is done.

Using the project created earlier, replace the default *TextView* with the following statements to the *main.xml* file.

main.xml

```
....  
  
<ImageView android:id="@+id/imageView"  
    android:layout_width="210dp"  
    android:layout_height="270dp"  
    app:layout_constraintLeft_toLeftOf="parent"  
    app:layout_constraintRight_toRightOf="parent"  
    app:srcCompat="@android:color/darker_gray"  
    app:layout_constraintTop_toTopOf="parent"  
    app:layout_constraintBottom_toBottomOf="parent" />  
....
```

Java Code

```
....  
  
public class MainActivity extends AppCompatActivity {  
  
    ImageView img;  
    final private int REQUEST_INTERNET = 123;  
  
    private InputStream OpenHttpConnection(String urlString) throws IOException  
    {  
        InputStream in = null;  
        int response = -1;  
  
        URL url = new URL(urlString);  
        URLConnection conn = url.openConnection();  
  
        if (!(conn instanceof HttpURLConnection))  
            throw new IOException("Not an HTTP connection");  
        try{  
            HttpURLConnection httpConn = (HttpURLConnection) conn;  
            httpConn.setAllowUserInteraction(false);  
            httpConn.setInstanceFollowRedirects(true);  
            httpConn.setRequestMethod("GET");  
            httpConn.connect();
```

```

        response = httpConn.getResponseCode();
        if (response == HttpURLConnection.HTTP_OK) {
            in = httpConn.getInputStream();
        }
    }
    catch (Exception ex)
    {
        Log.d("Networking", ex.getLocalizedMessage());
        throw new IOException("Error connecting");
    }
    return in;
}

private Bitmap DownloadImage(String URL)
{
    Bitmap bitmap = null;
    InputStream in = null;
    try {
        in = OpenHttpConnection(URL);
        bitmap = BitmapFactory.decodeStream(in);
        in.close();
    } catch (IOException e1) {
        Log.d("NetworkingActivity", e1.getLocalizedMessage());
    }
    return bitmap;
}

private class DownloadImageTask extends AsyncTask<String, Void, Bitmap> {
    protected Bitmap doInBackground(String... urls) {
        return DownloadImage(urls[0]);
    }

    protected void onPostExecute(Bitmap result) {
        ImageView img = (ImageView) findViewById(R.id.imageView);
        img.setImageBitmap(result);
    }
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    if (ContextCompat.checkSelfPermission(this,
        Manifest.permission.INTERNET)
        != PackageManager.PERMISSION_GRANTED) {

        ActivityCompat.requestPermissions(this,
            new String[] {Manifest.permission.INTERNET},
            REQUEST_INTERNET);
    }
}

```

```

    } else{
        new DownloadImageTask().execute("http://www.elcivics.com/
        transportation_corvette_car.jpg\n");
    }

}
@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions,
int[] grantResults) {
    switch (requestCode) {
        case REQUEST_INTERNET:
            if (grantResults[0] == PackageManager.PERMISSION_GRANTED) {

                new DownloadImageTask().execute("http://www.elcivics.com/
                transportation_corvette_car.jpg\n");

            } else {
                Toast.makeText(MainActivity.this, "Permission Denied",
                Toast.LENGTH_SHORT).show();
            }
            break;
        default:
            super.onRequestPermissionsResult(requestCode, permissions,
            grantResults);
        }
    }
}
}
....

```



How does it work?

The *DownloadImage()* method takes the URL of the image to download and then opens the connection to the server using the *OpenHttpConnection()* method that you have defined earlier. Using the *InputStream* object returned by the connection, the

decodeStream() method from the *BitmapFactory* class is used to download and decode the data into a *Bitmap* object. The *DownloadImage()* method returns a *Bitmap* object.

To download an image and display it on the activity, you call the *DownloadImage()* method. The *DownloadImage()* method is *synchronous*—that is, it will not return control until the image is downloaded—calling it directly freezes the UI of your activity. This is not allowed in Android 3.0 and later. All synchronous code must be wrapped using an *AsyncTask* class. Using *AsyncTask* enables you to perform background tasks in a separate thread and then return the result in a UI thread. That way, you can perform background operations without needing to handle complex threading issues.

To call the *DownloadImage()* method *asynchronously*, you need to *wrap* the code in a subclass of the *AsyncTask* class, as shown here:

```
private class DownloadImageTask extends AsyncTask<String, Void,
Bitmap> {
    protected Bitmap doInBackground(String... urls) {
        return DownloadImage(urls[0]);
    }

    protected void onPostExecute(Bitmap result) {
        ImageView img = (ImageView) findViewById(R.id.imageView);
        img.setImageBitmap(result);
    }
}
```

Basically, you define a class (*DownloadImageTask*) that *extends* the *AsyncTask* class. In this case, there are *two* methods within the *DownloadImageTask* class: *doInBackground()* and *onPostExecute()*.

You put all the code that needs to be run *asynchronously* in the *doInBackground()* method. When the task is completed, the result is passed back via the *onPostExecute()* method.

ImageView is used to display the downloaded image.

3. Downloading Text Content

Besides downloading binary data, you can also download plain-text content. For example, you might want to access a web service that returns a string of random quotes. The following code illustrates how you can download a string from a web service in your application.

You can modify the code in the previous example.

Java Code

```
....

public class MainActivity extends AppCompatActivity {

    final private int REQUEST_INTERNET = 123;

    private InputStream OpenHttpConnection(String urlString) throws IOException
    {
        InputStream in = null;
        int response = -1;

        URL url = new URL(urlString);
        URLConnection conn = url.openConnection();

        if (!(conn instanceof HttpURLConnection))
            throw new IOException("Not an HTTP connection");
        try{
            HttpURLConnection httpConn = (HttpURLConnection) conn;
            httpConn.setAllowUserInteraction(false);
            httpConn.setInstanceFollowRedirects(true);
            httpConn.setRequestMethod("GET");
            httpConn.connect();
            response = httpConn.getResponseCode();
            if (response == HttpURLConnection.HTTP_OK) {
                in = httpConn.getInputStream();
            }
        }
        catch (Exception ex)
        {
            Log.d("Networking", ex.getLocalizedMessage());
            throw new IOException("Error connecting");
        }
        return in;
    }

    private String DownloadText(String URL)
    {
        int BUFFER_SIZE = 2000;
        InputStream in = null;
        try {
            in = OpenHttpConnection(URL);
        } catch (IOException e) {
            Log.d("Networking", e.getLocalizedMessage());
            return "";
        }
    }
}
```

```

        InputStreamReader isr = new InputStreamReader(in);
        int charRead;
        String str = "";
        char[] inputBuffer = new char[BUFFER_SIZE];
        try {
            while ((charRead = isr.read(inputBuffer)) > 0) {
                //---convert the chars to a String---
                String readString =
                    String.valueOf(inputBuffer, 0, charRead);
                str += readString;
                inputBuffer = new char[BUFFER_SIZE];
            }
            in.close();
        } catch (IOException e) {
            Log.d("Networking", e.getLocalizedMessage());
            return "";
        }
        return str;
    }

    private class DownloadTextTask extends AsyncTask<String, Void, String> {
        protected String doInBackground(String... urls) {
            return DownloadText(urls[0]);
        }

        @Override
        protected void onPostExecute(String result) {
            Toast.makeText(getBaseContext(), result, Toast.LENGTH_LONG).show();
        }
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        if (ContextCompat.checkSelfPermission(this,
            Manifest.permission.INTERNET)
            != PackageManager.PERMISSION_GRANTED) {

            ActivityCompat.requestPermissions(this,
                new String[] {Manifest.permission.INTERNET},
                REQUEST_INTERNET);

        } else {
            new DownloadTextTask().execute("http://jfdimarzio.com/test.htm");
        }
    }

```

```

    }

}

@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions,
int[] grantResults) {
    switch (requestCode) {
        case REQUEST_INTERNET:
            if (grantResults[0] == PackageManager.PERMISSION_GRANTED) {

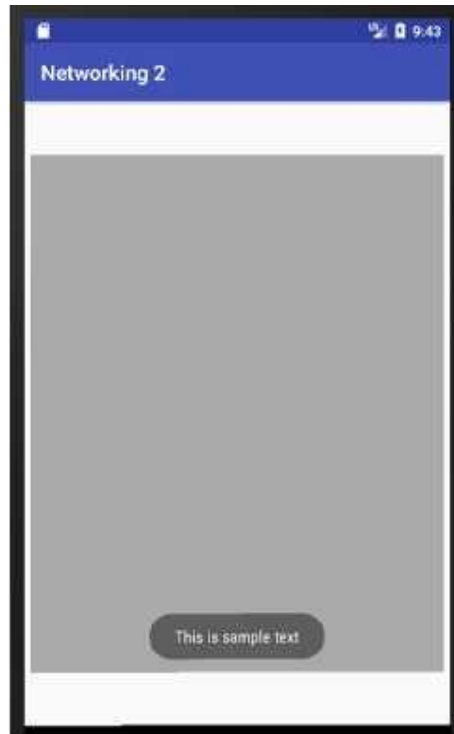
                new DownloadTextTask().execute("http://jfdimarzio.com/test.htm");

            } else {
                Toast.makeText(MainActivity.this, "Permission Denied",
                    Toast.LENGTH_SHORT).show();
            }
            break;
        default:
            super.onRequestPermissionsResult(requestCode, permissions,
                grantResults);
    }
}
}
....

```

How does it work?

The *DownloadText()* method accesses the text file's URL, downloads the text file, and then returns the desired string of text. It basically opens an HTTP connection to the server and then uses an *InputStreamReader* object to read each character from the stream and save it in a *String* object. As shown in the previously, you had to create a subclass of the *AsyncTask* class to call the *DownloadText()* method *asynchronously*.



4. Accessing Web Services Using the GET Method

Sometimes you may need to download XML files and parse the contents (a good example of this is consuming web services). In this section you learn how to connect to a web service using the HTTP GET method. After the web service returns a result in XML, you extract the relevant parts and display its content using the *Toast* class.

In this example, the web method from *http://services.aonaware.com/DictService/DictService.aspx?op=Define* is used. This web method is from a *dictionary web service* that returns the definition of a given word.

The web method takes a request in the following format:

```
GET /DictService/DictService.aspx/Define?word=string HTTP/1.1
Host: services.aonaware.com HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8 Content-Length: length
```

It returns a response in the following format:

```
<?xml version="1.0" encoding="utf-8"?>
<WordDefinition xmlns="http://services.aonaware.com/webservices/">
  <Word>string</Word>
  <Definitions>
    <Definition>
      <Word>string</Word>
      <Dictionary>
        <Id>string</Id>
        <Name>string</Name>
      </Dictionary>
      <WordDefinition>string</WordDefinition>
    </Definition>
  </Definitions>
</WordDefinition>
```

```

    <Word>string</Word>
    <Dictionary>
      <Id>string</Id>
      <Name>string</Name>
    </Dictionary>
    <WordDefinition>string</WordDefinition>
  </Definition>
</Definitions>
</WordDefinition>

```

Hence, to obtain the definition of a word, you need to establish an HTTP connection to the web method and then parse the XML result that is returned. The following code shows you how.

You can modify the code in the previous example.

Java Code

```

....

public class MainActivity extends AppCompatActivity {

    final private int REQUEST_INTERNET = 123;

    private InputStream OpenHttpConnection(String urlString) throws IOException
    {
        InputStream in = null;
        int response = -1;

        URL url = new URL(urlString);
        URLConnection conn = url.openConnection();

        if (!(conn instanceof HttpURLConnection))
            throw new IOException("Not an HTTP connection");
        try{
            HttpURLConnection httpConn = (HttpURLConnection) conn;
            httpConn.setAllowUserInteraction(false);
            httpConn.setInstanceFollowRedirects(true);
            httpConn.setRequestMethod("GET");
            httpConn.connect();
            response = httpConn.getResponseCode();
            if (response == HttpURLConnection.HTTP_OK) {
                in = httpConn.getInputStream();
            }
        }
        catch (Exception ex)
        {
            Log.d("Networking", ex.getLocalizedMessage());
            throw new IOException("Error connecting");
        }
        return in;
    }
}

```

```

private String WordDefinition(String word) {
    InputStream in = null;
    String strDefinition = "";
    try {
        in = OpenHttpConnection("http://services.aonaware.com/DictService/
            DictService.asmx/Define?word=" + word);
        Document doc = null;
        DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
        DocumentBuilder db;
        try {
            db = dbf.newDocumentBuilder();
            doc = db.parse(in);
        } catch (ParserConfigurationException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        } catch (Exception e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        doc.getDocumentElement().normalize();

        //---retrieve all the <Definition> elements---
        NodeList definitionElements =
            doc.getElementsByTagName("Definition");

        //---iterate through each <Definition> elements---
        for (int i = 0; i < definitionElements.getLength(); i++) {
            Node itemNode = definitionElements.item(i);
            if (itemNode.getNodeType() == Node.ELEMENT_NODE)
            {
                //---convert the Definition node into an Element---
                Element definitionElement = (Element) itemNode;

                //---get all the <WordDefinition> elements under
                // the <Definition> element---
                NodeList wordDefinitionElements =
                    (definitionElement).getElementsByTagName(
                        "WordDefinition");

                strDefinition = "";
                //---iterate through each <WordDefinition> elements---
                for (int j = 0; j < wordDefinitionElements.getLength(); j++) {
                    //---convert a <WordDefinition> node into an Element---
                    Element wordDefinitionElement =
                        (Element) wordDefinitionElements.item(j);

```

```

        //---get all the child nodes under the
        // <WordDefinition> element---
        NodeList textNodes =
            ((Node) wordDefinitionElement).getChildNodes();

        strDefinition +=
            ((Node) textNodes.item(0)).getNodeValue() + ". \n";
    }

    }
} catch (IOException e1) {
    Log.d("NetworkingActivity", e1.getLocalizedMessage());
}
//---return the definitions of the word---
return strDefinition;
}

private class AccessWebServiceTask extends AsyncTask<String, Void, String> {
    protected String doInBackground(String... urls) {
        return WordDefinition(urls[0]);
    }

    protected void onPostExecute(String result) {
        Toast.makeText(getBaseContext(), result, Toast.LENGTH_LONG).show();
    }
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    if (ContextCompat.checkSelfPermission(this,
        Manifest.permission.INTERNET)
        != PackageManager.PERMISSION_GRANTED) {

        ActivityCompat.requestPermissions(this,
            new String[] {Manifest.permission.INTERNET},
            REQUEST_INTERNET);

    } else{
        new AccessWebServiceTask().execute("computer");
    }
}

```

```

@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions,
                                     int[] grantResults) {

    switch (requestCode) {
        case REQUEST_INTERNET:
            if (grantResults[0] == PackageManager.PERMISSION_GRANTED) {

                new AccessWebServiceTask().execute("computer");

            } else {
                Toast.makeText(MainActivity.this, "Permission Denied",
                               Toast.LENGTH_LONG).show();
            }
            break;
        default:
            super.onRequestPermissionsResult(requestCode, permissions,
                                             grantResults);
    }
}
}
....

```



How does it work?

The *WordDefinition()* method first opens an HTTP connection to the web service, passing in the word for which you want the definition.

```
in = OpenHttpConnection("http://services.aonaware.com/DictService/  
DictService.asmx/Define?word=" + word);
```

DocumentBuilderFactory and *DocumentBuilder* objects are used to obtain a Document (DOM) object from an XML file (which is the XML result returned by the web service)

When the Document object is obtained, you find all the elements with the *<Definition>* tag.

Since the definition of a word is contained within the *<WordDefinition>* element, all the definitions are extracted within the loop.

The text content of the *<WordDefinition>* element contains the definition of a word, and the definitions of a word are then concatenated and returned by the *WordDefinition()* method.

A subclass of the *AsyncTask* class is used to call the *WordDefinition()* method asynchronously.

Finally, the web service is accessed asynchronously using the *execute()* method.

Consuming JSON services

Manipulating XML documents is a computationally expensive operation for mobile devices, for the following reasons:

XML documents are lengthy. They use tags to embed information, and the size of an XML document can pretty quickly become large. A large XML document means that your device must use more bandwidth to download it, which translates into higher cost.

XML documents are more difficult to process. When using *DocumentBuilderFactory*, you must use DOM to traverse the tree in order to locate the information you want. In addition, DOM itself has to build the entire document in memory as a tree structure before you can traverse it. This is both memory and CPU intensive.

A much more efficient way to represent information exists in the form of JSON (JavaScript Object Notation). JSON is a lightweight data-interchange format that is easy for humans to read and write. It is also easy for machines to parse and generate. The following lines of code show what a JSON message looks like:

```
[  
  {
```

```

        "appeId": "1",
        "survId": "1",
        "location": "",
        "surveyDate": "2008-03 14",
        "surveyTime": "12:19:47",
        "inputUserId": "1",
        "inputTime": "2008-03-14 12:21:51",
        "modifyTime": "0000-00-00 00:00:00"
    },
    {
        "appeId": "2",
        "survId": "32",
        "location": "",
        "surveyDate": "2008-03-14",
        "surveyTime": "22:43:09",
        "inputUserId": "32",
        "inputTime": "2008-03-14 22:43:37",
        "modifyTime": "0000-00-00 00:00:00"
    },
    {
        "appeId": "3",
        "survId": "32",
        "location": "",
        "surveyDate": "2008-03-15",
        "surveyTime": "07:59:33",
        "inputUserId": "32",
        "inputTime": "2008-03-15 08:00:44",
        "modifyTime": "0000-00-00 00:00:00"
    },
    {
        "appeId": "4",
        "survId": "1",
        "location": "",
        "surveyDate": "2008-03-15",
        "surveyTime": "10:45:42",
        "inputUserId": "1",
        "inputTime": "2008-03-15 10:46:04",
        "modifyTime": "0000-00-00 00:00:00"
    },
    {
        "appeId": "5",
        "survId": "32",
        "location": "",
        "surveyDate": "2008-03-16",
        "surveyTime": "08:04:49",
        "inputUserId": "32",

```

```

        "inputTime":"2008-03-16 08:05:26", \
        "modifyTime":"0000-00-00 00:00:00"
    }
]

```

The preceding block of lines represents a set of data taken for a survey. Note that the information is represented as a collection of *key/value* pairs, and that each *key/value* pair is grouped into an ordered list of objects. Unlike XML, there are no lengthy tag names. Instead, there are only brackets and braces.

The following code demonstrates how to process JSON messages easily using the `JSONArray` and `JSONObject` classes available in the Android SDK.

Java Code

```

....

public class MainActivity extends AppCompatActivity {
    public String readJSONFeed(String address) {
        URL url = null;
        try {
            url = new URL(address);
        } catch (MalformedURLException e) {
            e.printStackTrace();
        }
        StringBuilder stringBuilder = new StringBuilder();
        HttpURLConnection urlConnection = null;
        try {
            urlConnection = (HttpURLConnection) url.openConnection();
        } catch (IOException e) {
            e.printStackTrace();
        }
        try {
            InputStream content = new BufferedInputStream(
                urlConnection.getInputStream());
            BufferedReader reader = new BufferedReader( new
                InputStreamReader(content));
            String line;
            while ((line = reader.readLine()) != null) {
                stringBuilder.append(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            urlConnection.disconnect();
        }
        return stringBuilder.toString();
    }
}

```

```

    }

    private class ReadJSONFeedTask extends AsyncTask<String, Void, String> {

        protected String doInBackground(String... urls) {
            return readJSONFeed(urls[0]);
        }

        protected void onPostExecute(String result) {
            try {
                JSONArray jsonArray = new JSONArray(result);
                Log.i("JSON", "Number of surveys in feed: " + jsonArray.length());
                // print out the content of the json feed
                for (int i = 0; i < jsonArray.length(); i++) { JSONObject
                    jsonObject = jsonArray.getJSONObject(i);
                    Toast.makeText(getBaseContext(), jsonObject.getString("appeId") +
                        " - " + jsonObject.getString("inputTime"),
                        Toast.LENGTH_SHORT).show();
                }
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        new ReadJSONFeedTask().
            execute("http://extjs.org.cn/extjs/examples/grid/survey.html");
    }
}
....

```



How does it work?

The *readJSONFeed()* method connects to the specified URL and then reads the response from the web server. It returns a *string* as the result.

To call the *readJSONFeed()* method asynchronously, a subclass of the *AsyncTask* class is created,

The *readJSONFeed()* method is called in the *doInBackground()* method, and the JSON string that you fetch is passed in through the *onPostExecute()* method.

To obtain the list of objects in the JSON string, you use the *JSONArray* class, passing it the JSON feed as the constructor for the class:

```
JSONArray jsonArray = new JSONArray(result);
Log.i("JSON", "Number of surveys in feed: " + jsonArray.length());
```

The *length()* method returns the number of objects in the *jsonArray* object. With the list of objects stored in the *jsonArray* object, you iterate through it to obtain each object using the *getJSONObject()* method.

The *getJSONObject()* method returns an object of type *JSONObject*. To obtain the value of the *key/ value* pair stored inside the object, you use the *getString()* method (you can also use the *getInt()*, *getLong()*, and *getBoolean()* methods for other data types).

Finally, the JSON feed is accessed asynchronously using the *execute()* method:

```
new ReadJSONFeedTask().execute(
    "http://extjs.org.cn/extjs/examples/grid/survey.html");
```